

Stored Procedures

25 July 2026 18:39

```
ml() {  
  --  
  --  
  --  
  --  
  --  
}
```

```
ml()  
ml()  
ml()
```

Reusable

Database Programming

Procedure addRecodsInEmpPayment&Activitie():

Begin

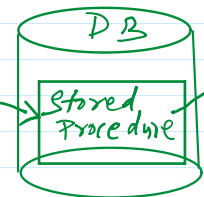
→ add one record in employee table
→ add one record in Payments table
→ add one record in Activity table

End

↑
best reusable
components

group of SQL
queries

Java App
Callable Statement



ORACLE → PL/SQL

MySQL → Procedure Language

Microsoft SQL Server → T-SQL (Transact SQL)

```
public int ml(int x, int y) {  
  int z = x + y;  
  return z;  
}
```

How many types of parameter stored procedure takes?

- ① IN parameter (To provide input value)
- ② OUT parameter (To collect output value)
- ③ INOUT parameter (To provide input values and to collect output values)

eg: $z := x + y \Rightarrow x \& y$ are IN parameter
 $\Rightarrow z$ is OUT parameter

$x := x + 1 \Rightarrow x$ is INOUT parameter

What is Stored Procedure?

In our programming if any code repeatedly required, then we can define that code inside a method and we can call that method multiple times based on our requirement.

Hence method is the best reusable component in our programming. Similarly in database programming, if any group of SQL statement

Hence procedure is the best reusable component in our programming. Similarly in database programming, if any group of SQL statement is repeatedly required then we can define those SQL statements in a single group repeatedly based on our requirement.

This group of SQL statements that performs a particular task is nothing but Stored Procedure. Hence stored procedure is the best reusable component at database level.

Hence, stored Procedure is a group of SQL statements that performs a particular task.

These procedures stored in database permanently for future purpose and hence the name stored procedure.

Usually stored procedure are created by Database Administrator (DBA). Every database has its own language to create stored procedures.

Stored procedure in JDBC (Java database connectivity)

A stored procedure is a precompiled collection of SQL statements stored inside the database. It performs one or more database operations and can accept input parameters, return output parameters, and even return result sets.

Instead of sending SQL queries from a Java program every time, the Java application simply calls the stored procedure.

In JDBC, stored procedures are executed using the **CallableStatement** interface.

Why use stored procedures?

Consider an application where the salary of an employee needs to be updated.

Without stored procedure

The Java program sends the SQL query every time.

```
SQL:
UPDATE employee
SET salary = salary + 5000
WHERE emp_id = 101;
```

The SQL statement is parsed, compiled, optimized and executed each time.

With Stored Procedure

The SQL statement is stored permanently inside the database.

```
SQL:
CALL increase_salary(101, 5000);
```

Only the procedure name and parameters are sent from Java, making execution faster.

Advantages of stored procedures

1. Faster execution because they are precompiled.
2. Reduces network traffic.
3. Improves security by hiding SQL queries.
4. Reusable by multiple applications.
5. Easier maintenance.
6. Reduces code duplication
7. Supports complex business logic.

How to create and execute stored procedure in Oracle?

```
create or replace procedure proc1(X IN number, Y IN number, Z OUT number) as
BEGIN
    Z:= X + Y
END
```

Notes:

SQL and PL, SQL are not case sensitive languages. We can use lower case and upper case also.

After writing stored procedure we have to compile. For this we require to use /(forward slash)

/ => compilation

While compiling if any error occur then we can check these errors by using the following command.

```
SQL> show errors;
```

Once we created stored procedure and compiled successfully, we have to register out parameters to hold result of stored procedure.

```
SQL> variable sum number; (declaring a variable)
```

We can execute the stored procedure with **execute** command as follows:

```
SQL> execute procedure(10, 100, sum);
```

```
SQL> print sum;
```

Practical implementation:

```
SQL> create or replace procedure p1(x IN number, y IN number, z OUT number) as
2  Begin
3      z:=x+y;
4  End
5  /
```

Warning: Procedure created with compilation errors.

```
SQL> show errors
```

Errors for PROCEDURE P1:

LINE/COL ERROR

```
-----
4/3      PLS-00103: Encountered the symbol "end-of-file" when expecting
```

one of the following:
; <an identifier> <a double-quoted delimited-identifier>
current delete exists prior <a single-quoted SQL string>
The symbol ";" was substituted for "end-of-file" to continue.

```
SQL> create or replace procedure p1(x IN number, y IN number, z OUT number) as
  2  Begin
  3      z:=x+y;
  4  End;
  5  /
```

Procedure created.

```
SQL> variable sum number;
SQL> execute p1(2.5, 7.9, sum);
BEGIN p1(2.5, 7.9, sum); END;
```

*

```
ERROR at Line 1:
ORA-06550: Line 1, column 20:
PLS-00201: identifier 'SUM' must be declared
ORA-06550: Line 1, column 7:
PL/SQL: Statement ignored
Help: https://docs.oracle.com/error-help/db/ora-06550/
```

```
SQL> execute p1(2.5, 7.9, :sum);
```

PL/SQL procedure successfully completed.

```
SQL> print sum
```

```
      SUM
-----
     10.4
```

```
SQL> execute p1(2.5, 7, :sum);
```

PL/SQL procedure successfully completed.

```
SQL> print sum
```

```
      SUM
-----
      9.5
```

```
CREATE OR REPLACE PROCEDURE GETSALARY
(
  EMPID IN VARCHAR2
, ESAL OUT NUMBER
) AS
BEGIN
  SELECT basic_salary into esal from employee WHERE emp_id = empid;
END GETSALARY;
```

Output:

Output Variables - Log	
Variable	Value
ESAL	35600

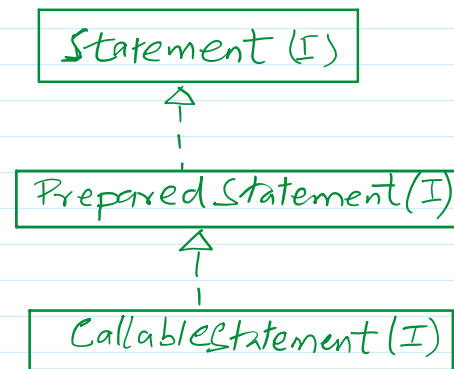
Java Code for calling stored procedure

Statement st = con.createStatement();

or

PreparedStatement ps = con.prepareStatement(SQLQuery);

CallableStatement cs = con.prepareCall(String procedure);



CallableStatement cs = con.prepareCall(" { call addProc(???) } ");

Annotations:

- JDBC Keyword points to `{`
- procedure name points to `addProc`
- parameters points to `(???)`
- it is check by DB. points to the entire string in quotes

public CallableStatement prepareCall (String procedureCall)
throws SQLException

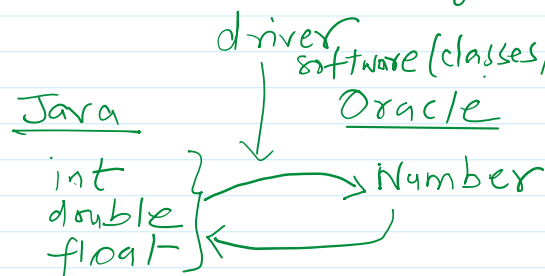
If we want to call the stored procedure in database by Java application:

CallableStatement (I) is implemented by driver software vendor.

```

public class Implement CallableInterface implements CallableStatement
{
    // ...
}
  
```

Mapping Java types to database types by using JDBC "Bridge" Types



String ← Varchar2

Java related data types and database related data types are not same. Some mechanism must be required to convert Java types to database types and from database types to Java types. This mechanism is nothing but "JDBC Types", which are known as "Bridge types".

Java data type	JDBC data type	Oracle data type
int	Types INTEGER	number
float	Types FLOAT	number
String	Types VARCHAR	varchar/varchar2.
Date	Types DATE	date.
⋮	⋮	⋮

Types is the class in java.sql package. INTEGER, FLOAT, VARCHAR, DATE etc. are static constant variable.

Process to call Stored Procedure from Java application by using CallableStatement.

1. Make sure Stored procedure already there in db.
2. Create CallableStatement object.
3. Provide values for every IN parameter by using corresponding setter methods
4. Register every OUT parameter with JDBC Type.
5. Execute procedure call.
6. get Result

6. get result

Converting to code.

1. Make sure Stored Procedure available in the database:

Create or replace procedure add (num1 IN number,
num2 IN number, result OUT number) as.
BEGIN
result := num1 + num2
END;
/

2. Create a CallableStatement with the procedure call.

CallableStatement cst = con.prepareStatement(" { call add(?, ?, ?) } ");

3. Provide values for every IN parameter by using corresponding setter methods.

```
cst.setInt(1, 100);  
cst.setInt(2, 100);
```

index value.

4. Register every OUT parameter with JDBC Types:

```
cst.registerOutParameter(3, Types.INTEGER);
```

index JDBC types

5. Execute Procedure call.

```
cst.execute();
```

6. Get result from OUT parameter by using corresponding
getXXX... () method.

```
int or = cst.getInt(3);  
out("Result: " + result);
```

```
package MyJDBCOne;  
import java.sql.CallableStatement;  
import java.sql.Connection;  
import java.sql.DriverManager;  
import java.sql.SQLException;
```

```

import java.sql.Types;
import java.util.Scanner;
public class StoredProcedureDemo1 {
    public static void main(String[] args) {
        String dbUrl = "jdbc:oracle:thin:@//localhost:1521/FREE";
        String username = "sys as sysdba";
        String password = "Calcutta1983";
        Scanner sc = new Scanner(System.in);

        try {
            Connection con = DriverManager.getConnection(dbUrl, username, password);
            CallableStatement cst = con.prepareCall("{call getsalary(?,?)}");
            System.out.println("Enter employee id: ");
            String eid = sc.next();
            cst.setString(1, eid);
            cst.registerOutParameter(2, Types.DOUBLE);
            cst.execute();
            System.out.println("Basic Salary: " + cst.getDouble(2));
            con.close();
            sc.close();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}

```

Output:

```

Enter employee id:
E006
Basic Salary: 70000.0

```